

Performance Testing

Date	06 July 2026
Team ID	TMID-SL-2026-01
Project Name	SmartLender - Applicant Credibility Prediction for Loan Approval
Maximum Marks	5 Marks

Performance testing evaluates how the system behaves under expected load conditions.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Flask test client + Python (time/statistics modules)
Type of Testing	Load testing (sequential requests) on the /submit prediction endpoint
Target Module / API	POST /submit — form submission → prediction → result render
Test Environment	Local (single-process Flask dev server via test client)
Test Date	06 July 2026

Step 2: Test Scenarios

S.No	Test Scenario	No. of Requests	Duration (sec)	Expected Outcome
1	Single valid applicant submission	1	< 0.1	200 OK, Approved/Rejected result rendered
2	Repeated sequential submissions (same payload)	100	< 2	All 200 OK, consistent prediction, no errors
3	Model + scaler cold load on app start	1	< 1	Model and scaler load without exception

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status	Remarks
1	Response Time (Avg)	< 300 ms	2.79 ms	Pass	100 sequential POST /submit requests
2	Response Time (p95)	< 500 ms	2.99 ms	Pass	95th percentile over 100 requests
3	Response Time (Max)	< 1000 ms	17.72 ms	Pass	Includes first-request warm-up
4	Error Rate	< 1%	0%	Pass	0 of 100 requests failed
5	Throughput	> 20 req/sec	~ 358 req/sec	Pass	Derived from average response time (single process, local)

Step 4: Observations & Analysis

The prediction endpoint is fast because inference is a single forward pass through a shallow (max_depth=4) XGBoost model on 11 scaled features — there is no external network call or database lookup in the request path. The model and scaler are loaded once at process start, so per-request cost is limited to form parsing, encoding, scaling, and inference. The single higher outlier (17.72 ms) corresponds to the first request after process start (interpreter/JIT warm-up) and is not representative of steady-state latency. These figures were measured against Flask’s local test client rather than over a network, so real-world latency under IBM Cloud hosting would additionally include network round-trip time.

Step 5: Screenshots / Evidence

Benchmark script: `Training/../Flask`, 100 sequential calls to `client.post('/submit', ...)` using Python's `time.perf_counter()`; raw results — avg 2.79 ms, p95 2.99 ms, max 17.72 ms, 0 errors across 100 requests.